```
!nvidia-smi
```

```
Tue May  5 05:04:06 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   39C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
# Import necessary libraries
import os
import zipfile
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# 1. Download Dataset
# Make sure you have the Kaggle API configured before running this!
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia

# 2. Extract Dataset
with zipfile.ZipFile("chest-xray-pneumonia.zip", "r") as zip_ref:
    zip_ref.extractall("chest_xray_dataset")

# Define paths to train, validation, and test sets
train_dir = "chest_xray_dataset/chest_xray/train"
val_dir = "chest_xray_dataset/chest_xray/val"
test_dir = "chest_xray_dataset/chest_xray/test"

# 3. Data Preprocessing
# Create data generators for augmentation and normalization
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,  # Normalize pixel values
    rotation_range=20,   # Augmentations
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator(rescale=1.0 / 255) # Only normalization for validation set

# Generate batches of image data
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224), # Resize images for VGG16
    batch_size=32,
    class_mode='binary'    # Binary classification
)

val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(224, 224)
```

```
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

# 4. Build the Model
# Load VGG16 pre-trained on ImageNet, excluding top layers
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze all convolutional layers
for layer in base_model.layers:
    layer.trainable = False

# Add custom dense layers for binary classification
x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x) # Binary classification: sigmoid activation

# Combine base model and new dense layers
model = Model(inputs=base_model.input, outputs=output)

# 5. Compile the Model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy', # Binary classification loss
    metrics=['accuracy']
)

# Print the model summary
model.summary()

# 6. Train the Model
model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=3,
    steps_per_epoch=len(train_generator),
    validation_steps=len(val_generator)
)

# 7. Evaluate the Model
test_datagen = ImageDataGenerator(rescale=1.0 / 255)
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

# Evaluate on test data
test_loss, test_accuracy = model.evaluate(test_generator)
print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_accuracy:.4f}")
```

```
Dataset URL: https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia
License(s): other
chest-xray-pneumonia.zip: Skipping, found more recently modified local copy (use --force to force dow
Found 5216 images belonging to 2 classes.
Found 16 images belonging to 2 classes.
Model: "functional_1"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten_1 (Flatten)	(None, 25088)	0
dense_2 (Dense)	(None, 256)	6,422,784
dense_3 (Dense)	(None, 1)	257

```
 Total params: 21,137,729 (80.63 MB)
 Trainable params: 6,423,041 (24.50 MB)
 Non-trainable params: 14,714,688 (56.13 MB)
Epoch 1/3
163/163 ━━━━━━━━━━━━━━━━━━━━ 122s 730ms/step - accuracy: 0.8769 - loss: 0.3550 - val_accuracy: 0.8125
Epoch 2/3
163/163 ━━━━━━━━━━━━━━━━━━━━ 112s 685ms/step - accuracy: 0.9360 - loss: 0.1558 - val_accuracy: 0.6875
Epoch 3/3
163/163 ━━━━━━━━━━━━━━━━━━━━ 112s 685ms/step - accuracy: 0.9331 - loss: 0.1643 - val_accuracy: 0.8750
Found 624 images belonging to 2 classes.
20/20 ━━━━━━━━━━━━━━━━━━━━ 7s 303ms/step - accuracy: 0.9022 - loss: 0.2580
Test Loss: 0.2580, Test Accuracy: 0.9022
```

```
# Import necessary libraries
import os
import zipfile
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
# 1. Download Dataset
# Make sure you have the Kaggle API configured before running this!
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia

# 2. Extract Dataset
with zipfile.ZipFile("chest-xray-pneumonia.zip", "r") as zip_ref:
    zip_ref.extractall("chest_xray_dataset")

# Define paths to train, validation, and test sets
train_dir = "chest_xray_dataset/chest_xray/train"
val_dir = "chest_xray_dataset/chest_xray/val"
test_dir = "chest_xray_dataset/chest_xray/test"

# 3. Data Preprocessing
# Create data generators for augmentation and normalization
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,  # Normalize pixel values
    rotation_range=20,   # Augmentations
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator(rescale=1.0 / 255) # Only normalization for validation set

# Generate batches of image data
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224), # Resize images for VGG16
    batch_size=32,
    class_mode='binary'    # Binary classification
)

val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

# 4. Build the Model
# Load VGG16 pre-trained on ImageNet, excluding top layers
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# No Freezing of Layers - Fine-tuning the whole model
for layer in base_model.layers:
    layer.trainable = True

# Add custom dense layers for binary classification
x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x) # Binary classification: sigmoid activation

# Combine base model and new dense layers
model = Model(inputs=base_model.input, outputs=output)

# 5. Compile the Model
# Using a smaller learning rate (1e-5) for fine-tuning
model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='binary_crossentropy', # Binary classification loss
    metrics=['accuracy']
)

# Print the model summary
model.summary()

# 6. Train the Model
```

```
model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=3,
    steps_per_epoch=len(train_generator),
    validation_steps=len(val_generator)
)

# 7. Evaluate the Model
test_datagen = ImageDataGenerator(rescale=1.0 / 255)
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

# Evaluate on test data
test_loss, test_accuracy = model.evaluate(test_generator)
print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_accuracy:.4f}")
```

```
Dataset URL: https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia
License(s): other
chest-xray-pneumonia.zip: Skipping, found more recently modified local copy (use --force to force downl
Found 5216 images belonging to 2 classes.
Found 16 images belonging to 2 classes.
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0